
GC-LDM: Genre-Conditioned Latent Diffusion for Controllable Music Generation

Ksheer Sagar Agrawal
UC San Diego
ksagrawal@ucsd.edu

Saachi Shenoy
UC San Diego
svshenoy@ucsd.edu

Avinash Gondela
UC San Diego
agondela@ucsd.edu

Abstract

We present **GC-LDM**, a genre-conditioned latent diffusion system for music generation. Our core design replaces heavyweight text-conditioning modules with a lightweight learned genre-token conditioner while keeping the proven latent-diffusion backbone. This design targets a practical product goal: *fast, controllable, single-GPU music generation* with explicit style control. Our current implementation uses FMA-small, AudioLDM’s pretrained VAE for latent compression, a genre-conditioned U-Net denoiser, and HiFi-GAN for waveform reconstruction. We report training and validation loss, perform several ablation studies on inference steps, CFG scale, genre consistency, quality-control and diversity and per-genre quality and efficiency along with a summary of metrics. Overall, results show a clear quality-control tradeoff: higher CFG improves genre accuracy at the cost of FAD proxy, while more inference steps improve FAD proxy at increased runtime.

1 Introduction & Project Scope

1.1 Product Vision: What research problem are we aiming to solve

The goal of this project is to develop a **genre-conditioned deep generative model** for music generation, where the input is a genre ID and the output is an original audio clip that reflects the stylistic traits of that genre. Generating music that captures genre-specific traits remains a challenging problem, as musical genres differ in rhythm, instrumentation, structure, tempo, and overall composition patterns. By conditioning the generative process on genre information, the model aims to learn the relationship between high-level genre labels and the underlying audio representations. This allows the system to generate music that follows the structural and stylistic conventions of different genres while still producing novel compositions. Such a system has practical applications including royalty-free background music for content creators, adaptive soundtracks in video games that respond to gameplay dynamics, and idea-generation tools for music producers and composers.

1.2 Motivation and Importance

Large text-to-music systems (5; 8) are powerful but computationally expensive and "heavy." The relevance of this problem shows up in its focus on control in deep generative models, rather than purely random generation. By allowing users to select a genre, the model provides direct guidance over the generation process, which is a core objective in many applied generative systems where user intent must guide generation, like domain-specific LLMs or interactive image generation models. Beyond the technical importance, this solution is also valuable in terms of its practical uses. While this model likely won’t replace the traditional audio generation pipeline in the music industry, it has applications in areas where music is a secondary aspect of the final product, such as video games or social media. This solution would also improve the accessibility of music generation, as it allows non-musicians to create content without technical or musical experience.

1.3 Existing Works and their Limitations

One of the main works in audio generation is WaveNet, which shows that autoregressive models are able to work directly with raw audio waves. But, because it uses sequential sampling, generation is slow and computationally expensive. It is also limited in the control the user has to specify audio style or structure. When we look past the foundational works and into more recent works, such as Jukebox, we see that autoregressive modeling conditioned on rich user-provided inputs can guide music generation. However, this conditioning is reliant on a complex and large-scale pipeline, making simple genre only control difficult to isolate and also making it inefficient for lightweight applications. This infrastructure requires massive datasets and very large models, meaning this is not a practical solution for most research settings and users. There are also diffusion-based audio models such as AudioLDM that solve the problem of resources by working on compressed audio and also support textual guidance. This model also relies on many pretrained components (e.g., CLAP, T5), limiting its reproducibility. All of these works show the battle between controllability, efficiency, and quality, and highlight the need for an accessible, understandable, and lightweight approach to audio generation.

2 Proposed Solution for solving this problem

To solve this problem, we design **GC-LDM**, a genre-conditioned deep generative model that generates short audio clips based on a specified genre input. The model is trained on a dataset of music samples labeled by genre, allowing it to learn the relationship between genre categories and their corresponding audio characteristics such as rhythm, structure, and instrumentation. During generation, the selected genre acts as a conditioning signal that guides the model to produce new audio consistent with the stylistic traits of that genre. This approach enables controllable and flexible music generation while maintaining diversity in the generated outputs along with the efficiency of generation unlike large music generation systems.

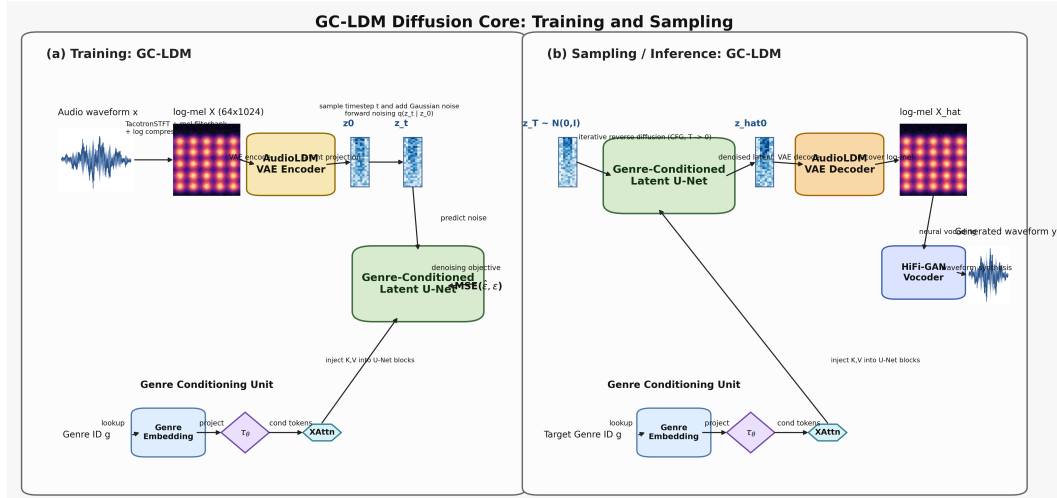


Figure 1: Architecture diagram of proposed solution which shows the overview of GC-LDM system for genre conditional generation.

3 Detailed Methods (Math, Model, Algorithm & Discussion of Proposed Method)

3.1 Mathematical Formulation for Dataset Curation

The goal of dataset curation is to transform the raw audio from the dataset into structured training samples suitable for conditional latent diffusion. Each curated sample is of the form (z', y) where z'

is the latent representation of an audio segment and y is the numerical encoding of its genre. We built deterministic track-level splits and a manifest to avoid leakage across slices from the same track.

Current cached latent dataset size:

- total latents: 23,982,
- total tracks: 7,994,
- train/val/test tracks: 6,394 / 800 / 800.

3.1.1 Audio Preprocessing

Each audio file is loaded as a mono waveform and resampled to a fixed rate

$$f_s = 16,000 \text{ Hz}.$$

Let the signal be $x[n]$. To ensure consistent amplitude statistics across the dataset, the waveform is normalized as:

$$x'[n] = x[n] - \frac{1}{N} \sum_{k=0}^{N-1} x[k], \quad x''[n] = \frac{x'[n]}{\max(|x'[n]|) + \varepsilon},$$

$$\tilde{x}[n] = 0.5 x''[n].$$

This normalization matches the distribution expected by the pretrained AudioLDM encoder.

3.1.2 Fixed-Length Segmentation

Audio is divided into uniform 10-second segments:

$$L = 10f_s = 160000 \text{ samples}.$$

Signals shorter than L are zero-padded and longer signals are truncated, producing fixed-size inputs suitable for batch training.

3.1.3 Latent Representation via Pretrained VAE

A pretrained AudioLDM variational autoencoder encodes the spectrogram into a latent distribution

$$q_\phi(z|Y) = \mathcal{N}(\mu_\phi(Y), \sigma_\phi^2(Y)).$$

For deterministic dataset construction, the posterior mode is used:

$$z = \mu_\phi(Y),$$

and scaled using the pretrained constant s :

$$z' = s z.$$

This step compresses high-dimensional audio into a structured latent space that preserves semantic acoustic information while greatly reducing dimensionality.

3.1.4 Genre Conditioning

Each track’s genre label g is mapped to an integer identifier

$$\mathcal{G}(g) = y, \quad y \in \{0, \dots, K-1\},$$

which serves as the conditioning signal for the generative model.

3.1.5 Final Curated Dataset

Each processed segment is stored as a pair

$$(z', y),$$

forming the dataset

$$\mathcal{D} = \{(z'_i, y_i)\}_{i=1}^N.$$

Operating in latent space instead of waveform space significantly reduces storage and computational requirements, enabling efficient training of a conditional diffusion model for genre-controlled audio generation.

3.2 Diffusion Overview

In a diffusion model, we start with a clean latent representation and then combine it with increasing amounts of Gaussian noise. We then train the model to predict and remove this noise. Subsequently, the generation process uses this method by starting with a random latent and attempting to denoise it. We apply diffusion in the latent space defined by the pretrained VAE $z_0 \in \mathbb{R}^{8 \times 256 \times 16}$ from the preprocessing step.

3.2.1 Denoising Overview

We parameterized the diffusion model with a neural network ϵ_θ that takes as input a latent z_t with noise, timestep t , and genre g , and outputs a prediction of the noise $\epsilon_\theta(z_t, t, g)$. The training objective uses the MSE of the sampled noise and the predicted noise: $L = \|\epsilon - \epsilon_\theta(z_t, t, g)\|_2^2$. Here, z_t , the noisy latent, is created by adding noise to z_0 , where t is the diffusion timestep, and g is the genre label that we use for conditioning. (5)

3.2.2 U-Net Architecture

AudioLDM implements a custom U-Net architecture within its latent diffusion framework. Rather than modifying that implementation, we use HuggingFace’s UNet2DConditionModel and set it up to mimic the cross-attention and structure used in AudioLDM. It is a standard U-Net structure with a downsampling path that is meant to capture the global structure, a bottleneck representation, and an upsampling path to reconstruct details. Cross-attention layers in the network let the genre g influence the denoising process at multiple scales (6). This means both the global structure and the local details are influenced by the genre. The model runs on latent tensors of size $[b, 8, 256, 16]$, where b is the batch size.

3.2.3 Genre Embedding and Token Projection

Instead of conditioning on text (as in AudioLDM), we condition on a genre ID. Each genre label $g \in \{0, \dots, 7\}$ is mapped to an embedding vector $e_g \in \mathbb{R}^{512}$. This embedding is trained with the parameters of the diffusion model. We also introduce a "null" genre to support classifier-free guidance during training and sampling. For the cross-attention layers we project the genre embedding into multiple tokens, $T_g \in \mathbb{R}^{8 \times 512}$ via a linear projection and reshaping.

3.2.4 Classifier-Free Guidance (CFG)

To strengthen genre control at generation, we implemented a strategy known as classifier-free guidance. In training, the model is exposed to conditioned (with genre) and unconditioned (with the null genre) inputs. This lets the network learn both conditioned and unconditioned denoising behaviors within the same model. Then, when sampling, we combine the two noise predictions as $\hat{\epsilon} = \epsilon_{\text{uncond}} + s \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$, where s is the guidance scale. Larger values of s increase the influence of the genre-conditioned noise, and if $s = 0$, then it defaults to unconditional generation. This approach is meant to improve the control of the condition (3).

3.3 Algorithm sketch

Training loop

1. Sample (z_0, g) from manifest dataloader.
2. Sample $t \sim \mathcal{U}\{1, \dots, T\}$, $\epsilon \sim \mathcal{N}(0, I)$.
3. Build z_t , predict $\epsilon_\theta(z_t, t, C_{\tilde{g}})$.
4. Minimize MSE loss; update with AdamW.
5. Periodically run validation and save latest/best checkpoints.

Sampling loop

1. Start from Gaussian latent z_T .
2. Run reverse scheduler with CFG and target genre ID.

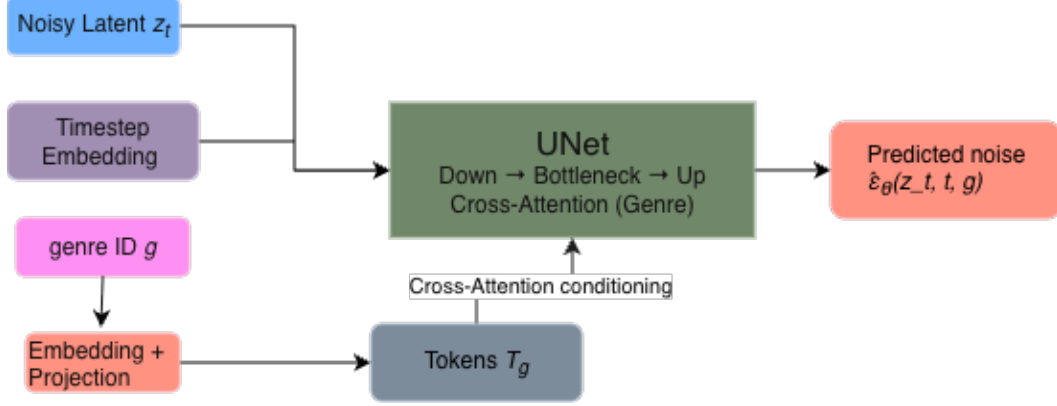


Figure 2: Genre-conditioned U-Net architecture detail.

3. Decode final latent with VAE decoder.
4. Vocode with pretrained HiFi-GAN (4).

3.4 Final implementation status

Component	Status	Notes
Latent preprocessing (FMA-small, z +genre ID)	Done	23,982 cached latents
Deterministic splits + manifest + dataloader	Done	Track-level leakage control
Genre conditioning module + U-Net wiring	Done	Text path replaced with genre tokens
Train loop + checkpoint + resume	Done	Best/latest checkpoints
Sampling script (genre ID + CFG + seed)	Done	Latent/audio export path
Full convergence + objective evaluation	Done (proxy)	Loss, FAD proxy, classifier metrics

Table 1: Overall progress snapshot.

4 Evaluation Metrics

We evaluate the model with a mix of loss-based diagnostics and audio-level proxies. Since we do not have human listening tests, we use lightweight proxy metrics and report trends:

- **Training/validation loss** and **validation noise MSE**: mean squared error between predicted and true noise on held-out latents.
- **Genre consistency (proxy)**: external classifier accuracy and confusion matrix using AST + keyword mapping.
- **Audio quality (proxy)**: log-mel Fréchet distance between real and generated audio distributions (lower is better).
- **Diversity**: intra-genre vs inter-genre latent distance statistics (higher inter-genre than intra-genre indicates separation).
- **Efficiency**: inference step count as a runtime proxy.

5 Experiments and Results

5.1 Experimental Settings

We trained the diffusion model using the AdamW optimizer with a learning rate of 1×10^{-4} , a weight decay of 1×10^{-2} and batch size of 16 for 10 epochs. Additionally gradient clipping was applied for stability, and a cosine annealing schedule was used to vary the learning rate. The objective is mean squared error between the predicted and ground-truth noise and classifier-free guidance is incorporated during training via conditioning dropout with probability 0.1. In diffusion, we used a DDPM scheduler with 1000 timesteps. During inference and ablation studies, we vary the number of denoising steps and the classifier-free guidance (CFG) while keeping the rest of the parameters stable. Validation was performed every 500 steps on up to 100 batches, and model checkpoints are saved periodically. The best model is selected according to validation loss. All experiments are conducted on a GPU with a fixed random seed of 42 to ensure reproducibility.

5.2 Training Loss Curves

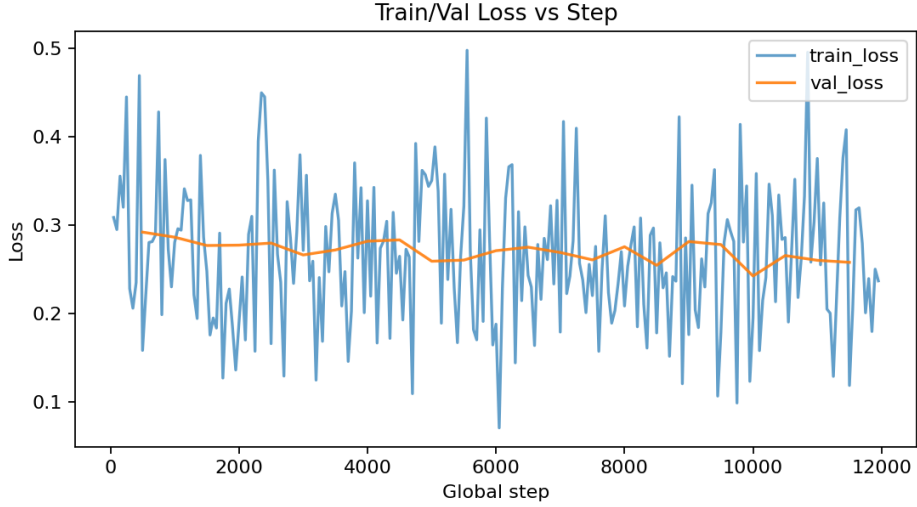


Figure 3: Training and validation loss vs step.

Validation loss decreases over training and the held-out validation noise MSE reaches 0.258, indicating improved denoising performance on unseen latents. Training loss remains noisy due to stochastic noise sampling but does not diverge, suggesting stable optimization.

5.3 Ablation 1: Inference Steps

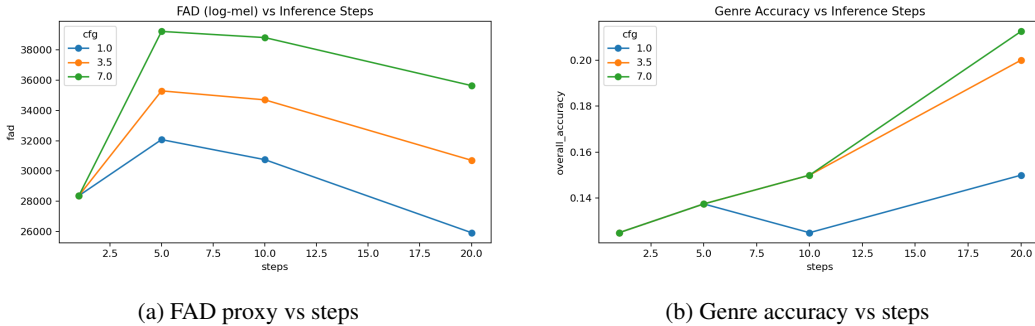


Figure 4: Effect of inference steps on quality and control.

FAD proxy decreases as inference steps increase, with the best FAD proxy of 25934 at CFG=1.0 and 20 steps. Genre accuracy also improves with steps, peaking at 0.213 for CFG=7.0 and 20 steps (above the random baseline of 0.125). This indicates more steps improve both quality and control, at the expected runtime cost.

5.4 Ablation 2: CFG Scale

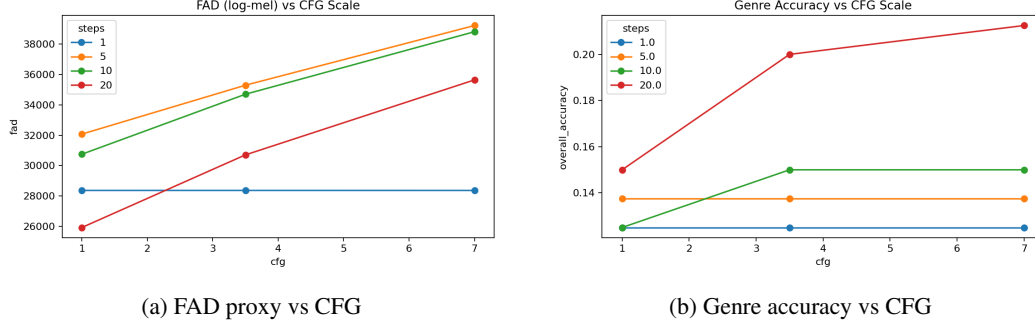


Figure 5: Effect of CFG scale on quality and control.

Genre accuracy increases with CFG scale, reaching 0.213 at CFG=7.0 and 20 steps, while the FAD proxy worsens relative to 25934 at CFG=1.0 and 20 steps. This shows stronger conditioning improves control but trades off audio quality.

Each ablation setting uses 10 samples per genre (8 genres), totaling 80 samples per combo across 3 CFG scales and 4 inference step counts.

5.5 Genre Consistency

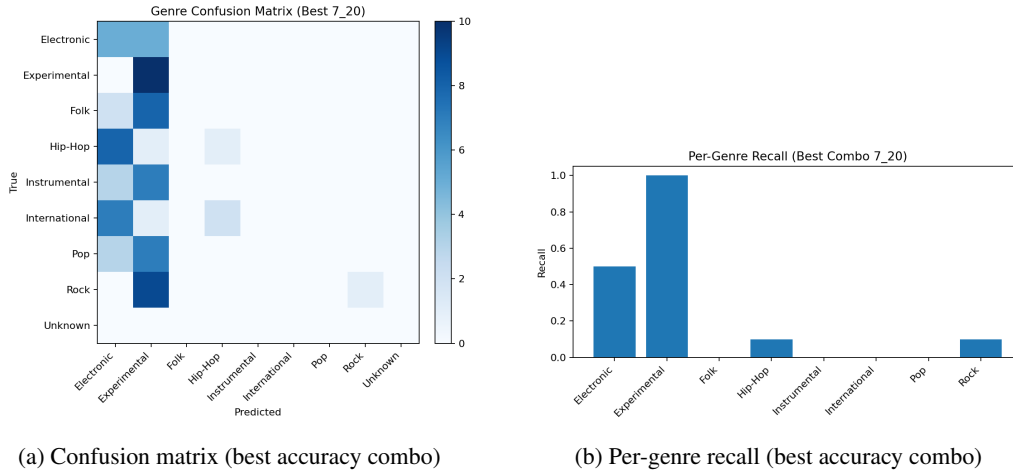


Figure 6: External classifier-based genre consistency (proxy). Here 7_20 refers to the pair (CFG scale, inference steps).

The best accuracy combo is CFG=7.0 with 20 steps (7_20), achieving 0.213 overall genre accuracy, above the 0.125 random baseline. However, the confusion matrix shows substantial cross-genre confusion, indicating limited separability. Electronic and Hip-Hop show the highest per-genre recall, while Folk and International are most frequently misclassified.

5.6 Quality-Control and Diversity

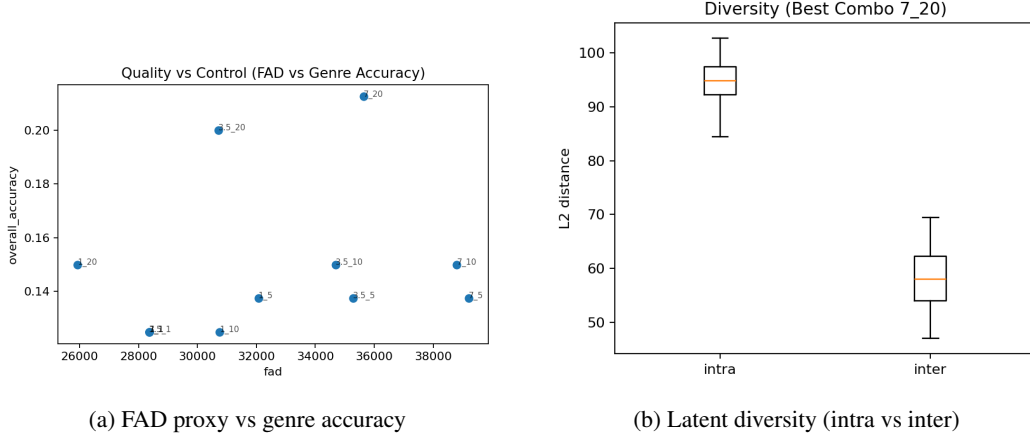


Figure 7: Quality-control tradeoff and diversity statistics. Here 7_20 refers to the pair (CFG scale, inference steps).

The scatter plot shows that stronger control (higher genre accuracy) comes with a higher (worse) FAD proxy; the best-accuracy combo has FAD 35632. Latent diversity at the best-accuracy combo shows intra-genre mean ~ 94 and inter-genre mean ~ 58 , indicating overlap rather than strong separation.

5.7 Per-Genre Quality and Efficiency

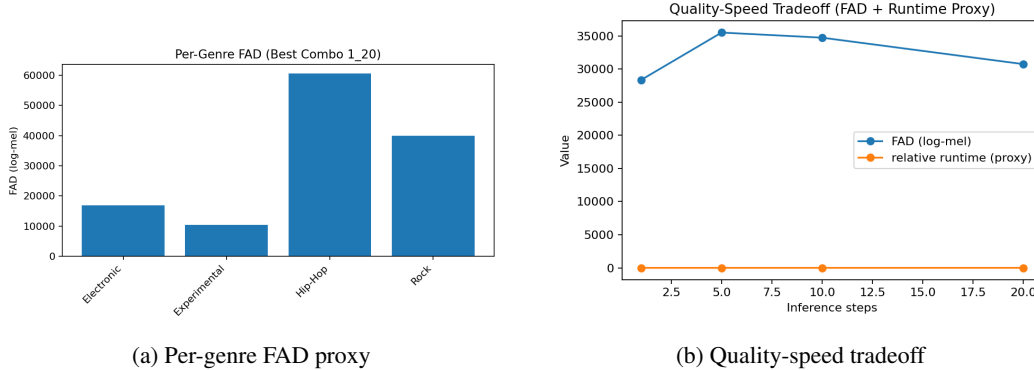


Figure 8: Per-genre quality and runtime proxy. Here 1_20 refers to the pair (CFG scale, inference steps).

Per-genre FAD proxy varies substantially across genres, with Experimental and Rock showing the highest FAD values under the best overall combo (CFG=1.0, 20 steps, FAD=25934), suggesting these genres are harder to model faithfully. The quality-speed plot confirms runtime grows linearly with step count, making it the primary efficiency knob — users targeting faster inference should prefer 5–10 steps with only modest quality loss.

6 Discussion: Novelty, Advantages, and Limitations

Novelty and Significance The primary novelty of this work lies in the development of a genre-conditioned deep generative framework that balances fine-grained control with computational efficiency. While large-scale architectures have successfully demonstrated text-to-music synthesis,

these systems typically rely on massive proprietary datasets and prohibitive computational resources. Furthermore, prior research often entangles genre control with complex textual prompts or latent representations that are difficult to isolate. Our approach proposes a lightweight, explicit conditioning mechanism that makes controllable music generation accessible within resource-constrained academic environments.

Advantages over Existing Works The first advantage we have is control simplicity: using an explicit genre provides clear conditioning, and avoids the ambiguity of complex prompts. The second is compute efficiency: by removing large text encoders such as CLAP or T5 we reduce memory overhead and computational usage. Finally, we use a pretrained encoder/decoder which isolates the training problem to the diffusion model and conditioning, simplifying optimization and reproducibility.

Limitations While GC-LDM has many practical attributes, there are limitations to our methodology and the model components themselves. Regarding the data, training the model on 10-second clips limits its ability to learn genre-structural components that span the length of a piece of music. Without hierarchical conditioning, it does not explicitly model long-form temporal dependencies, such as the relationship between the verse and chorus. Additionally, some of the genres have lyrics while others are mainly instrumental, and the combination of lyrics and music potentially inhibited learning in those genres. We found that the model had a harder time generating genres like rap, where speech and lyrical content are defining characteristics. Another point of future optimization could be how we evaluate our model. Our evaluation metrics are proxy-based as we use log-mel FAD and classifier-based genre consistency, both of which approximate quality but do not fully encapsulate a human listening experience. Finally, our methodology has a strong dependence on pretrained components such as the AudioLDM VAE and HiFi-GAN, meaning that in the future, we could work on creating custom components that are closer to our use case.

7 Conclusion

GC-LDM demonstrates practical, controllable music generation with lightweight conditioning and a realistic training stack. Our architecture replaces complicated text conditioning with simple genre selection which allows for controllable and efficient training. The results of our experimentation show our diffusion model achieved stable training and that the model was able to learn representations of music successfully. We also conducted ablation experiments that showcase the tradeoff between genre controllability and audio quality, both of which are influenced by classifier-free guidance scale and diffusion step limit. Overall, the results show that lightweight conditional diffusion models can be used for efficient and controllable music generation without large-scale infrastructure. In future work, this project could be expanded by training over larger datasets, improving evaluation metrics, and handling modeling of long-term musical structure.

7.1 Broader Impact

Potential positive uses include royalty-free creative tooling, content creation support, fast prototyping for musicians, and accessible music creation for non-musicians. Systems like GC-LDM could help creators in generating background music for videogames, social media posts, or pieces of media where music is a background component instead of the main focus. Additionally, the model could be used as an educational tool, allowing users to explore the characteristics of different musical genres by generating examples.

Potential risks include misuse for copyright circumvention, over-reliance on automated creativity, and misuse in commercial settings. Since generative models learn from existing musical data, there is a risk that our generated outputs could unintentionally be modeled after existing works or have elements mimicking particular artists.

GitHub Code: <https://github.com/ksheersagaragrawal/GC-LDM>

References

- [1] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson. FMA: A Dataset for Music Analysis. In *ISMIR*, 2017.
- [2] J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. In *NeurIPS*, 2020.
- [3] J. Ho and T. Salimans. Classifier-Free Diffusion Guidance. *arXiv:2207.12598*, 2022.
- [4] J. Kong, J. Kim, and J. Bae. HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis. In *NeurIPS*, 2020.
- [5] H. Liu et al. AudioLDM: Text-to-Audio Generation with Latent Diffusion Models. In *ICML*, 2023.
- [6] Hugging Face Diffusers Documentation: UNet2DConditionModel. <https://huggingface.co/docs/diffusers/main/api/models/unet2d-cond>.
- [7] R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. In *CVPR*, 2022.
- [8] A. Agostinelli et al. MusicLM: Generating Music From Text. *arXiv:2301.11325*, 2023.
- [9] P. Dhariwal et al. Jukebox: A Generative Model for Music. *arXiv:2005.00341*, 2020.
- [10] P. Zuo et al. MeLoDy: Efficient Neural Music Generation. *arXiv:2305.15719*, 2023.